

Instructions That Never Arrive: Measuring Whether Operator Compaction Instructions Reach the Summarizer in a Production LLM Coding Agent

Anton Dziatkovskii 

Platinum Software Development Company, Brisbane, Australia

ORCID 0000-0001-7408-3054 dzyatkovskiy.a2@gmail.com <https://github.com/tonydz>

September 2026

Abstract

Long-running LLM coding agents survive finite context windows by *compaction*: replacing the conversation with a model-written summary. Compaction is lossy, and practitioners are widely advised to steer it by writing “compact instructions” into a project configuration file or a pre-compaction hook. We measure whether those instructions arrive. On two machines of one production fleet we scanned complete transcript archives and, for every compaction event, recovered what the operator actually passed to the command and scored the resulting summary for the operator’s custom seven-header skeleton. On machine A (16,107 transcript files; 456 events; CLI 2.1.161–2.1.246) not one of 447 compactations run without an inline instruction produced the skeleton. On machine B, measured for this paper (10,435 files; 469 events; CLI 2.1.149–2.1.260), 3 of 431 did (0.7%, 95% CI 0.2–2.0%) — and in all three the header text was already in the conversation before compaction, consistent with format carry-over rather than instruction delivery. Passing the same instructions *inline*, the documented path, produced a top-level skeleton in 16 of 38 runs on machine B (42%; 16–23 of 38 once partial applications are counted) and 5 of 9 on machine A: often, not deterministically. We do not pool the two, one operator and one shared configuration being no basis for an independence claim. A seeded probe on the current release carries the causal half: unique marker headers written into the configuration file scored 0 of 7, the same markers pasted inline 7 of 7 with 15 of 15 seeded facts surviving. We also report, with fixtures, four defects that make a naive version of this measurement report successes that are not there: our first, substring-based detector calls 82 of 335 bare compactations successes where the corrected one finds 2. The study is single-fleet, Windows-only, one agent product; the detector, the fixtures and the full per-event table ship with the paper so the numbers can be contested.

1 Introduction

An LLM coding agent that runs for hours hits the context window long before the task ends. The dominant remedy is *compaction*: the harness asks a model to summarize the conversation, then continues from that summary. Compaction is lossy by construction and not lossy at random — the first casualties are *rationales* (why an alternative was rejected) and *exact values* (the actual path, the actual identifier), exactly what a resumed session needs.

Harnesses therefore offer, or appear to offer, ways for the operator to steer the summarizer: a section in the project instruction file, a pre-compaction hook, a settings key. This paper asks one checkable question: *when an operator writes compaction instructions where they are told to write*

them, do those instructions reach the summarizer? For one widely used production agent the answer is essentially no. Across 925 compaction events on two machines, spanning client releases 2.1.149–2.1.260, instructions written anywhere but inline with the command left no measurable trace. The mechanism that does work — passing them as command arguments — is the documented one, and it lands in roughly two to three runs out of five.

We claim no novelty for the mechanism. The contributions are: a measured negative result with a stated denominator, replicated on a second machine whose numbers disagree with the first instructively (§5); *format carry-over* as the explanation for that disagreement — a summary can reproduce the skeleton simply because the skeleton sits in the conversation as content, so a study that does not check for it over-counts successes; four defects in the naive measurement, each with a fixture that turns red on the defective rule (§6) — the part we expect to transfer, being properties of append-only transcripts rather than of one product; and operating guidance drawn from the numbers (§8).

2 System under measurement

The subject is Claude Code, a command-line coding agent, versions 2.1.149–2.1.260 (June–September 2026) on Windows; it writes an append-only JSONL transcript per session under `~/.claude/projects/`. Four operator paths matter: **bare** `/compact`, which summarizes with the harness’s own prompt; **inline arguments** (`/compact <text>`), which pass that text to the summarizing call and are the documented steering mechanism; **automatic compaction**, which fires when the window fills and accepts nothing from the operator; and **out-of-band configuration** — a `CLAUDE.md` section widely recommended in community write-ups, a `PreCompact` hook, a settings key mentioned in blog posts — which is the path under test. Two upstream issues bear on it: #14160, requesting custom instructions for automatic compaction, closed as duplicate, notes the hook receives an *empty* custom-instructions field on that trigger; #43733, requesting instruction injection from the hook, closed *not planned* — the hook can print to stderr and block, but cannot steer.¹

The format under test is a seven-header preservation order, published as **compact-canon**: DECISIONS (with rejected alternatives and why), TODO, NOW, PATHS & VALUES, COUNTERS, OPEN, TOOLS & CONTRACTS, in an English and a Russian variant; both are scored. The order encodes what dies first, which is why rationales and exact values get their own mandatory headers.

3 Related work

Compaction is the practical face of a studied problem. Memory architectures move state out of the window — MemGPT/Letta pages tiered memory with self-editing [1], A-MEM organizes agent memory as linked notes [2], Zep maintains a temporal knowledge graph [3] — and position-dependent recall [4] argues against treating a long window as a substitute for deliberate preservation. Coding-agent interface design is active [5], but the interface *to the summarizer* appears undocumented: nobody measures whether an operator’s preservation policy reaches the summarizing call. We claim **no novelty** for summarization-based context management, for the observation that summaries lose rationales, or for passing instructions inline — the documented behaviour. The contribution is the measurement, its failure modes, and the replication.

¹Issues <https://github.com/anthropics/claude-code/issues/14160> and <https://github.com/anthropics/claude-code/issues/43733>. We did not test the settings key: we found no documentation for it.

4 Method

For each transcript file we parse every record and emit one row per compaction event: trigger, what the operator passed, and the score of the resulting summary.

Identifying an event. A compaction produces a summary record flagged `isCompactSummary` and a *sibling* record carrying `compactMetadata` (trigger, pre/post token counts, anchors), joined through `compactMetadata.preservedMessages.anchorUuid`, which names the summary’s UUID. Where the anchor is absent (15 of 469 events) we fall back to the latest summary at or before the metadata timestamp and flag the row.

Recovering the arguments. The invocation is a user record containing `<command-name>/compact</command-name>` and `<command-args>`; each summary is paired with the nearest such record at or before its timestamp. Rows are bucketed *auto*, *bare* (empty arguments), *free-text* (arguments that are not the block), *block* (≥ 6 of the 7 headers in the arguments).

Scoring. A header counts only if it *opens a line*, allowing at most one leading space and an optional bullet or heading marker. We compute three scores per summary — this strict rule, a tolerant line-opening rule permitting arbitrary indentation, and plain substring presence — because the difference between them is itself a result (§6). A summary “produced the skeleton” when all seven headers score strictly.

Carry-over check. For every summary scoring seven we re-scan the file for an earlier non-summary record containing ≥ 6 of the header words; if one exists, the format was visible to the summarizer as ordinary content and the row is flagged. For block runs the flag is trivially true — the pasted block is such a record — so it is informative only for instruction-free buckets. Proportions carry Wilson score 95% intervals, zero-numerator results the rule-of-three bound. The detector is deterministic Python with no model call and ships with the fixtures of §6.

5 Results

Machine A is the fleet hub: 16,107 transcript files, 456 compaction events with joined metadata, CLI 2.1.161–2.1.246, 2026-06-06 to 2026-08-29, scanned with the corrected detector and published in the companion repository. **Machine B** is a second node of the same fleet, same operator configuration, wider band of releases, scanned for this paper: 10,435 transcript files, 469 events after de-duplication, CLI 2.1.149–2.1.260, 2026-05-29 to 2026-09-09. Table 1 gives both.

Bucket	Machine A		Machine B			
	<i>n</i>	skeleton	<i>n</i>	substring	strict	carry-over
automatic compaction	39	0	78	1	1	1
bare /compact	380	0	335	82	2	2
/compact, other free text	28	0	18	4	0	—
without an inline block	447	0	431	87	3	3
/compact with the block inline	9	5	38	31	16	(n/a)

Table 1: Both archives. “Strict” is the corrected detector; “substring” is the defective one, kept for comparison (machine A’s published figure for that defect is 67 of 405). “Carry-over” counts strict hits whose header text was already in the conversation before compaction. Machine A: 0 of 447 gives 95% CI 0–0.9%, rule-of-three bound 0.67%.

Three findings follow. (i) *The negative result survives replication, but it is not exactly zero.* Three of 431 instruction-free compactations (0.70%, 95% CI 0.24–2.03%) produced the full skeleton where machine A found none in 447. The archives do not conflict — 0.7% sits inside machine A’s

interval — but reporting machine A alone would have stated a cleaner claim than the evidence supports.

(ii) *All three are carry-over — which constrains, but does not prove, the mechanism.* Every strict hit in the instruction-free buckets had the header text already present before compaction, and all three used the Russian header variant the operator’s own blocks used. Inspection agrees: in one case a block had been pasted 90 minutes earlier in the same session; in another the header words appear in records from the previous day; the third is an *automatic* compaction, a trigger that structurally cannot receive operator instructions. We state the limit plainly: carry-over is an association, and post hoc we cannot separate *imitation of visible formatting* from *retention of an instruction given earlier in the conversation*. The causal claim about the configuration-file path therefore rests on the seeded nonce-marker probe below, not on these three events; the archive supports only the weaker statement that without an inline block the skeleton is essentially absent, and never appears where its text was not already visible.

(iii) *The inline path is frequent, not deterministic.* On machine B the block produced a top-level skeleton in 16 of 38 runs (42.1%, 95% CI 27.9–57.8%); machine A reported 5 of 9. We deliberately do *not* pool these: one operator, shared configuration, overlapping calendar windows, so a combined interval would assert an independence we cannot claim. The strict rule is a lower bound here. Inspecting all 7 block runs the tolerant rule accepts and the strict rule rejects, every one carries the session’s real content under all seven headers, indented beneath the stock template — the block partially applying rather than an unrelated quotation. Read as a band, the inline path lands in 16–23 of 38 runs (42–61%). A run that misses entirely returns the plain stock summary. For scale, the median event compressed 265,778 tokens to 14,449 ($\approx 18\times$).

Sensitivity and provenance. Fifteen events needed the timestamp fallback rather than the anchor join, all in instruction-free buckets; excluding them leaves 3 of 416 (0.72%), so the result does not depend on the fallback. Machine B’s archive is local — the only project directories bearing another node’s path hold synced memory files and zero transcripts — so its 469 events are not a re-count of machine A’s.

Seeded live probes. Archive scans describe what happened but cannot isolate a cause. Three probes seeded a session with verifiable facts (paths, identifiers, three counters, one explicitly rejected decision) and then compacted. *Headless, CLI 2.1.186, 2026-07-23*: with the block inline, `trigger=manual`, 54,358 tokens in and 2,522 out, 7/7 headers, 15/15 spot-checked facts surviving including the counters and the rejected-decision rationale — and a correction of our own earlier belief, since the transcript showed the block *supplements* the stock template rather than replacing it, at roughly 2.5k duplicated tokens. *Interactive, same date*: a human pressed the command in a real session — 7/7 headers, 9/9 spot-check keys. *Current release, CLI 2.1.246, 2026-09-05*: unique marker headers placed in the configuration file’s compact-instructions section, then a bare compaction, scored **0 of 7** markers with 6 of 15 seeded facts surviving; the same markers pasted inline scored 7 of 7 and 15 of 15. Because the markers were strings that appeared nowhere else, this probe, unlike the archive counts, does separate delivery from carry-over — and it is the one we would rerun first on another harness. One release change matters for reproduction: on 2.1.246 a one-exchange session refuses to compact (**not enough messages to compact**), so the seed-then-compact-immediately protocol no longer runs as written; our probe needed five exchanges first.

6 Four defects that manufacture successes

Each defect below was found by our own measurement being wrong first, and each is a property of append-only agent transcripts rather than of this product.

D1. Substring matching is a false-positive machine. A stock summary that merely *quotes*

the instruction line (“args requested the 7-heading format: DECISIONS · TODO · ...”) contains all seven strings and scores seven. On machine B it calls 82 of 335 bare compactions successes where the strict rule finds 2: a 24% effect that does not exist.

D2. Line-opening is not enough either, and indentation is ambiguous. Requiring the header to open a line removes D1, but a summary that reproduces the headers as an *indented list* still scores seven: - ****DECISIONS:**** ... opens a line under any tolerant rule. On machine B this is the difference between 23 and 16 in the block bucket and between 3 and 2 in the bare bucket. The ambiguity is real and cuts both ways: indented headers in a *bare* run are the stock summary quoting a block it was shown (a false positive), while the same shape in a *block* run is the instruction partially applying under the stock template (a false negative for the strict rule). No regular expression separates those; we cap the leading whitespace for the headline number, publish both counts, and inspect the disagreements by hand.

D3. The transcript is not in timestamp order, and the metadata sits on a sibling record. The command record is written *after* the summary it caused, so pairing a summary with “the last command seen while reading the file” attaches the wrong arguments; and since `compactMetadata` is not on the summary record, a scan expecting both on one record classifies nothing at all. On machine A this bug made two block runs look like failures and five bare runs like successes before it was caught.

D4. Records are duplicated in resumed files. A file appended by a resumed run can hold the same record twice by UUID. Our first pass counted 534 events; de-duplicating gives 469. Uncorrected, this inflates whichever bucket the operator happens to resume into.

Fixtures. Three fixtures make this checkable in seconds: (A) a stock summary quoting the seven names inside a line; (B) one quoting the block as an indented list; (C) a genuine skeleton. The corrected rule must score 0, 0, 7; the substring rule scores 7, 7, 7 and the tolerant rule 0, 7, 7 — both fail, as they must. We ran the fixtures red on both defective rules before trusting any number in §5.

7 Threats to validity

One fleet, one product, one operating system. Two machines under one operator, Windows only, sharing a configuration; not independent samples, which is why nothing is pooled. **Post-hoc transcript forensics is a floor, not a count.** A successful compaction rewrites the session file, and there is a known failure mode in which a compaction reports success without applying [6]; that report observed orphaned summaries whose originating commands were gone. A clean archive can look identical to one with history erased: our numbers are what survives in the files, not necessarily what happened. **Small n , and a numerator noise can move.** The negative side rests on a numerator of 3, so a two- or three-event classification error — a missing command record, a mis-attached timestamp — moves the 0.7% materially; hence the sensitivity check and the nonce probe beside it. The Russian header set also contains ordinary words (“PATHS”, “OPEN”, “TOOLS”), which is precisely why the strict rule matters. Nothing was preregistered. **Self-measurement.** We built the format, then measured it; the incentive runs toward finding it works, and it was our own too-strong headline — 7/7 from a single live run, generalized — that a third-party maintainer challenged, prompting the corrective re-scan. We publish detector, fixtures and per-event table for that reason.

8 What an operator should do

1. **Assume the configuration file is not read by the summarizer** until measured otherwise on your own harness — and measure with unique markers, not words that could appear anyway.
2. **Pass the preservation order inline with the command**, filled with the session’s actual facts: the only path we can measure working, at the cost of one paste.
3. **Compact deliberately, at a task boundary, before the window fills**. Automatic compaction cannot be steered: every one is a decision made for you.
4. **Verify by interrogation, not appearance**. Ask afterwards why a specific alternative was rejected: a steered summary answers specifically, a stock one in generalities. At a landing rate near one in two this check is not optional.
5. **Do not use compaction as memory**. Whatever must survive the week belongs in a durable store outside the transcript.

9 Conclusion

Across 925 compaction events on two machines, spanning client releases 2.1.149–2.1.260, operator instructions written where the community recommends writing them left no measurable trace on the summarizer; the three apparent exceptions all occurred in sessions where the format was already visible as content. The documented inline path works, in two to three runs out of five. The transferable part is smaller than the headline and more useful: four ways such a measurement reports successes that are not there, each with a fixture that goes red.

Code, data and contact

Three artefacts ship with this preprint: `compact_detect.py` (the deterministic detector, no model call, read-only), `_test_compact_detect.py` (the three fixtures of §6, with flags that reproduce the defective rules going red), and `machineB_events.json` (all 469 events with bucket, all three scores, carry-over and join flags, client version and token counts; local file paths stripped). The measurement protocol and machine A’s data live in the repository: <https://github.com/tonydzi/compact-canon>. Companion tooling: <https://github.com/tonydzi/claw-retro> (prints the block at session close), <https://github.com/tonydzi/claw-consensus>, <https://github.com/tonydzi/second-brain-starter-kit>, <https://github.com/tonydzi/sqlite-graph-memory>. Lab and personal site: <https://github.com/tonydzi> · <https://tonydzi.github.io>. Companion preprints from the same fleet: *Operating a Human-Governed Multi-Machine LLM Agent Fleet: An Experience Report*, 10.5281/zenodo.22639714; *When Does Graph Expansion Help Personal-Corpus Retrieval?*, 10.5281/zenodo.22639718; *Homeostatic Governance*, 10.5281/zenodo.22639712; *Le Chatelier Tokenomics*, 10.5281/zenodo.22639710. Correspondence: dzyatkovskiy.a2@gmail.com · ORCID 0000-0001-7408-3054 · WhatsApp +1 (341) 222-9178. We hand free working seeds of this tooling to engineer-testers reproducing the measurement on their own archive; a disagreeing scan is more useful than a confirming one.

Author contribution and AI use

The measurements, the detector and this manuscript were produced by the author working with LLM agents under the author’s review and editorial control; the author is responsible for all claims. The corrections in §5–§6 follow an external maintainer’s challenge to an earlier, stronger headline.

References

- [1] C. Packer et al., “MemGPT: Towards LLMs as Operating Systems,” arXiv:2310.08560, 2023.
- [2] W. Xu et al., “A-MEM: Agentic Memory for LLM Agents,” arXiv:2502.12110, 2025.
- [3] P. Rasmussen et al., “Zep: A Temporal Knowledge Graph Architecture for Agent Memory,” arXiv:2501.13956, 2025.
- [4] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” arXiv:2307.03172, 2023.
- [5] J. Yang et al., “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” arXiv:2405.15793, 2024.
- [6] Claude Code issue #89040, “/compact reports success without applying,” <https://github.com/anthropics/claude-code/issues/89040>.